LAB - LECTURE 3

HTML + CSS

Layout: FLEXBOX & GRID

Contatto: **ellepuntopi.98@gmail.com**



Programma di oggi

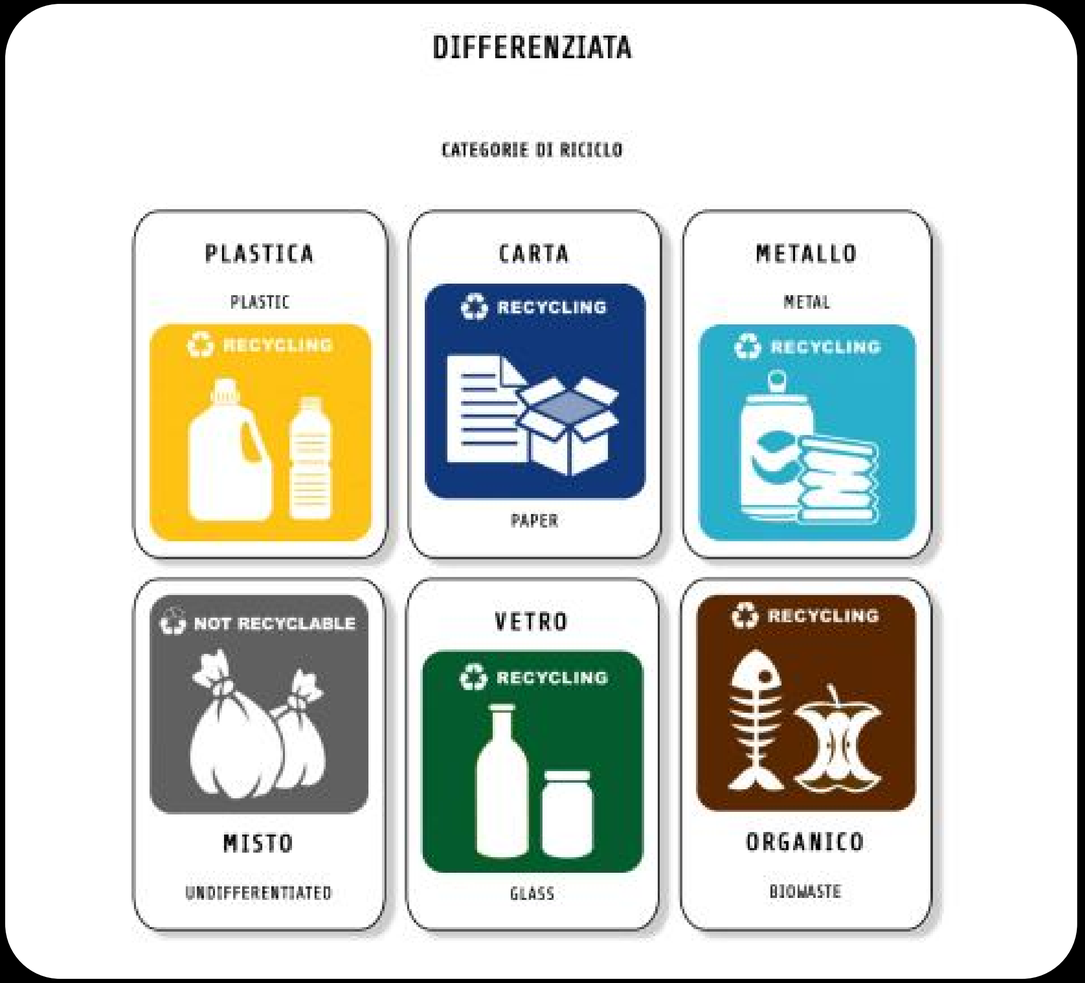
1. Esercizio Guidato
2. Esercizio Individuale
3. Introduzione alla Grid
4. Esercizio per casa

Esercizio Guidato - Raccolta Differenziata

Scrivere il codice HTML e CSS per visualizzare nel browser il contenuto qui a fianco.

In questo esercizio non siete liberi di fare come volete: dovremo utilizzare un **Layout FlexBox** per l'intero contenuto, **comprese le singole schede di riciclo**.

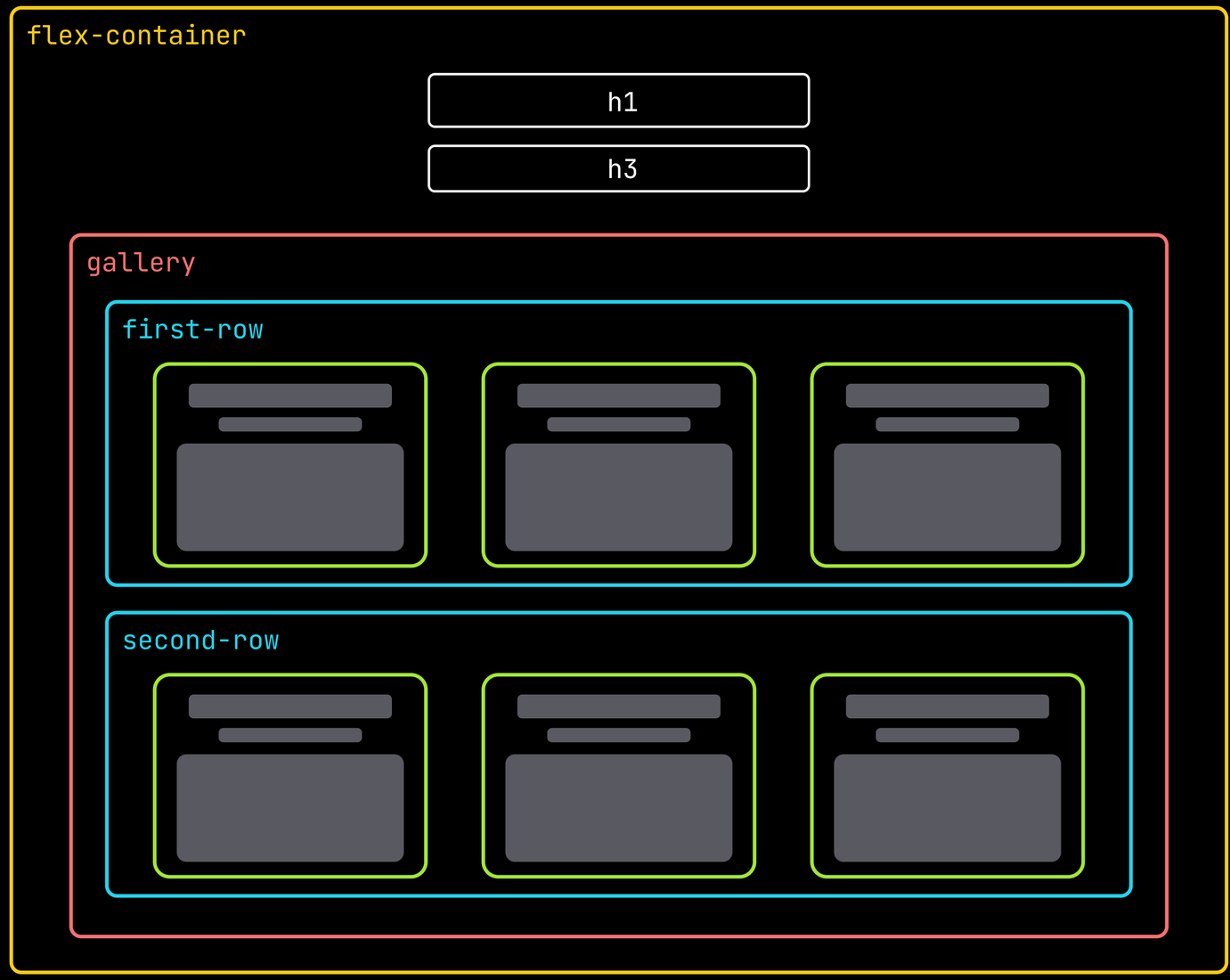
Come al solito è tutto sulla repo GitHub.



Esercizio Guidato (1) - Struttura dell'HTML

Come prima cosa ispezioniamo insieme l'HTML.
Partendo dal generale per arrivare allo specifico, il file *index_flex.html* è organizzato così:

- `flex-container`
- `h1, h3`
- `gallery`
- `first-row, second-row`
- `image-container`
- `title + subtitle + img`



Esercizio Guidato (2) - flex-container

Partiamo dall'elemento più "esterno" (il container più grande): **flex-container**.

```css
.flex-container {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 20px;
    width: 100%;
    max-width: 800px;
}
```

L'elemento ha la proprietà **display** con valore **flex**: stiamo impostando l'elemento per contenere al suo interno delle **flexbox**.

flex-direction organizza le box in colonna (**column**), mentre **align-items** posiziona al centro (**center**) ogni sotto-elemento contenuto al suo interno.

gap definisce lo spazio tra ogni elemento contenuto nel **flex-container**.

Esercizio Guidato (3) - gallery

Una volta organizzati in colonna **h1**, **h3** e **gallery**, entriamo nel dettaglio: organizziamo **gallery**, che contiene **first-row** e **second-row**.

```css
.gallery {
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    gap: 20px;
    width: 100%;
    max-width: 700px;
}
```

Di nuovo mettiamo in colonna i due sotto-elementi con **display: flex**. Questa volta aggiungiamo anche **justify-content**.

justify-content → asse **principale** (quello di flex-direction)
align-items → asse **secondario** (perpendicolare al principale)

Qui la direzione è **column**: justify-content centra in verticale, align-items in orizzontale.

Justify segue la direzione, align le va di traverso (perpendicolare).

Esercizio Guidato (4) - first-row e second-row

Ora posizioniamo gli elementi dentro **first-row** e **second-row**.

Di nuovo **display: flex**, ma questa volta la direzione è **row**: vogliamo le tre schede di riciclo di ogni riga una di fianco all'altra.

```
#first-row, #second-row {
    display: flex;
    flex-direction: row;
    flex-wrap: nowrap;
    gap: 20px;
    justify-content: space-around;
    width: 100%;
}
```

Con **row** l'asse principale diventa orizzontale: **justify-content** e **align-items** si scambiano!

flex-wrap decide se gli elementi possono andare a capo (**wrap**) o no (**nowrap**). Qui li vogliamo tutti e tre sulla stessa riga. Su schermi stretti è un problema: lo risolveremo con le *media-query*.

justify-content: space-around distribuisce lo spazio intorno ai **figli**, cioè alle schede della riga (non alle due righe).

Esercizio Guidato (5) - image-container

Siamo arrivati alle schede: gli elementi con classe **image-container**. Anche qui usiamo **display: flex** in **column**: titolo, sottotitolo e immagine uno sotto l'altro.

```css
.image-container {
    text-align: center;
    background-color: white;
    border: 2px solid black;
    padding: 15px;
    border-radius: 10%;
    display: flex;
    flex-direction: column;
    gap: 2px;
    box-shadow: 10px 8px 4px 0 rgba(0, 0, 0, 0.2);
}
```

Oltre a testo centrato, sfondo, bordo e padding:

border-radius: 10% → angoli arrotondati: 10% della larghezza (in orizzontale) e dell'altezza (in verticale) dell'elemento.

box-shadow: l'ombra della scheda

`10px`	spostamento x (+ destra, – sinistra)
`8px`	spostamento y (+ basso, – alto)
`4px`	sfocatura (0 = ombra netta)
`0`	espansione (+ allarga, – stringe)
`rgba(…, 0.2)`	colore: nero al 20%

Esercizio Guidato (6) - flex-grow, flex-shrink, flex-basis

Le schede sono anche **figli** delle righe flex: possiamo decidere come si dividono lo spazio della riga.

```
.image-container {
    /* ... */
    flex-grow: 1;
    flex-shrink: 1;
    flex-basis: 150px;
}

/* In una sola riga (grow - shrink - basis): */
/* flex: 1 1 150px; */
```

flex-grow → *peso* con cui la scheda si prende lo spazio avanzato. Tutte a 1 = parti uguali; se ne mettessimo una a 2, quella prenderebbe il doppio dello spazio *in più* rispetto ad una settata a 1.

flex-shrink → quanto la scheda *cede* rispetto alle altre quando lo spazio non basta (tutte a 1 = cedono uguale; 0 = non si stringe mai).

flex-basis → larghezza di *partenza*, prima di crescere o stringersi.

Attenzione: con “**flex-wrap: wrap**” e “**flex-grow: 1**”, la scheda che va a capo è da sola sulla riga e si prende tutto lo spazio, diventando enorme.
Per evitarlo: **flex-grow: 0** e una **flex-basis** sensata. Di solito però si gestisce con le *media-query* e/o con la *grid*.

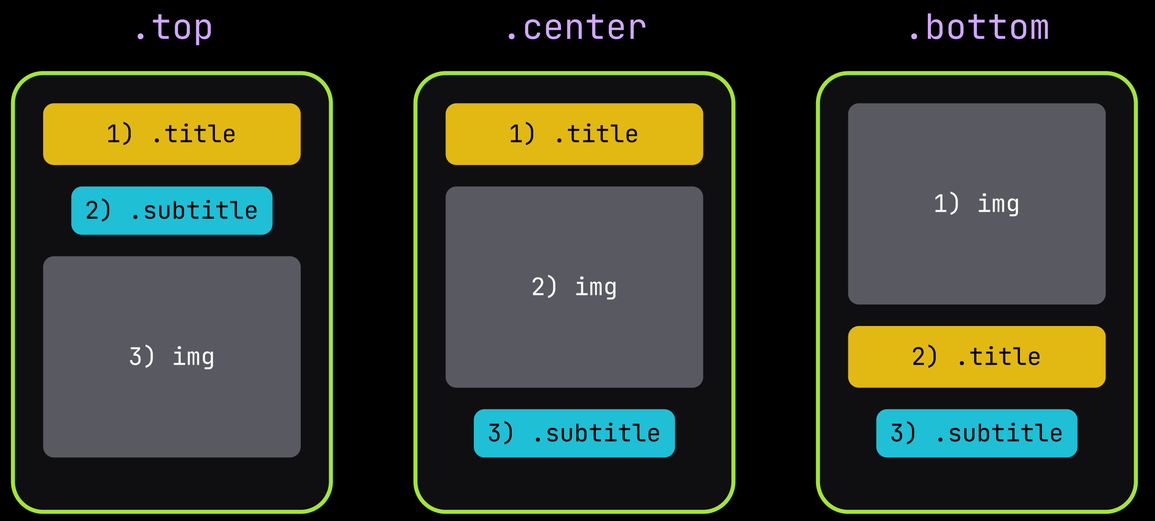
Esercizio Guidato (7) - order

Tutte le schede hanno lo stesso HTML (titolo, sottotitolo, immagine), ma li mostrano in ordini diversi. Con **order** decidiamo la posizione visiva di ogni figlio: dal valore più basso al più alto.

```css
.top .title       { order: 1; }
.top .subtitle    { order: 2; }
.top img          { order: 3; }

.center .title    { order: 1; }
.center img       { order: 2; }
.center .subtitle { order: 3; }

.bottom img       { order: 1; }
.bottom .title    { order: 2; }
.bottom .subtitle { order: 3; }
```



titolo → sottotitolo → immagine

titolo → immagine → sottotitolo

immagine → titolo → sottotitolo

N.B. Il default è **order: 0**: se diamo un order positivo (es. order: 1) a un solo elemento, questo finisce in fondo, perché gli altri, con 0, vengono prima. L'HTML non cambia, poiché cambia solo l'ordine *visivo*.

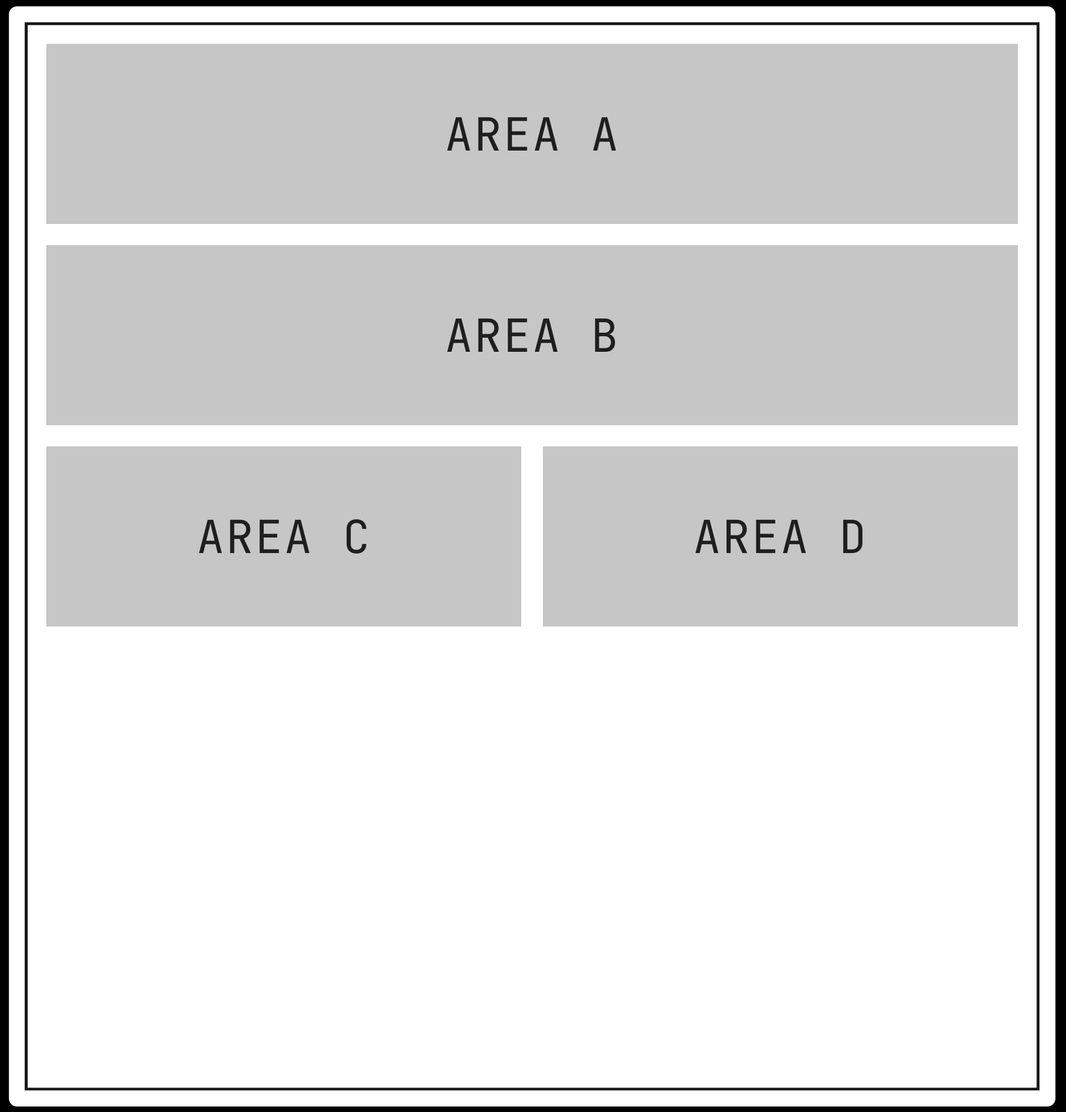
I **selettori sono *nested***: prima la classe della scheda (**.top**), poi l'elemento (**.title**, **.subtitle**, **img**).

Esercizio Individuale

Partendo dal codice HTML che trovate su GitHub, ricostruite il layout qui a fianco tramite un **layout flex**:

- **no grid**
- **no modifiche all'HTML**

Lavorate da soli: poi lo correggiamo insieme.



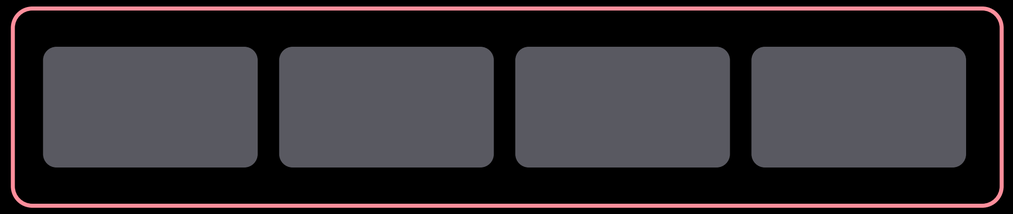
Introduzione alle Grids



CSS Grid: layout a due dimensioni

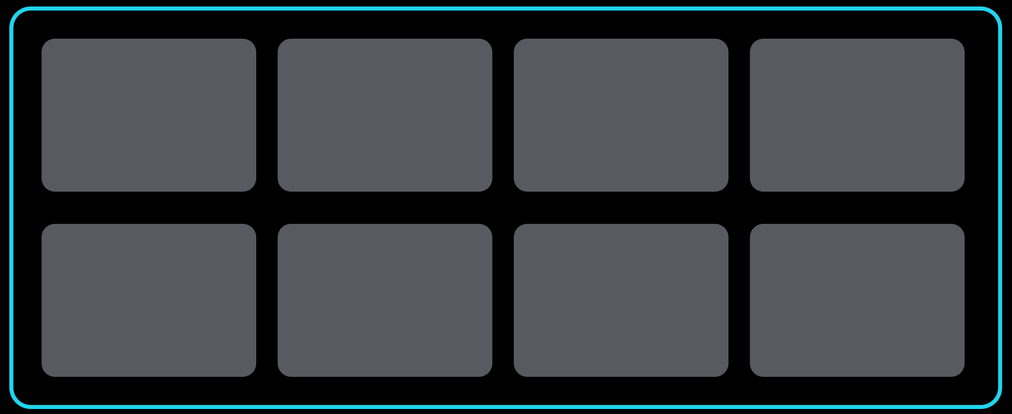
Il **flexbox** dispone gli elementi lungo **un solo asse** (riga *oppure* colonna).
La **grid** li organizza su **righe e colonne insieme**, come una tabella.

`display: flex`



→ un asse: gli elementi in fila

`display: grid`



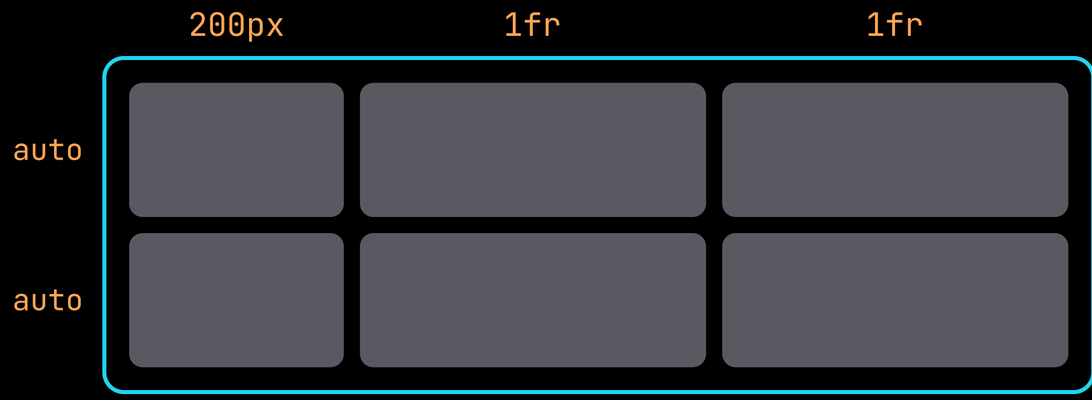
→ righe e colonne: ogni elemento ha la sua cella

Quando usarla? Per i layout “a griglia”: struttura della pagina (header, sidebar, contenuto, footer), gallerie, dashboard.
Il flexbox resta perfetto per allineare elementi in fila (menu, bottoni, card in una riga).
Restano comunque due modi di organizzare il layout in CSS, e sono quasi sempre interscambiabili.

Definire la griglia

Si parte dal **contenitore**: con **display: grid** i suoi figli diventano celle della griglia.

```
.container {
    display: grid;
    grid-template-columns: 200px 1fr 1fr;
    grid-template-rows: auto auto;
    gap: 10px;
}
```



→ 3 colonne × 2 righe = 6 celle. La prima è larga 200px, le altre due si dividono in parti uguali lo spazio che resta.

grid-template-columns → una misura per ogni colonna: quante colonne e quanto sono larghe.

grid-template-rows → una misura per ogni riga: ne decide l'altezza (**auto** = si adattano al contenuto).

fr → una parte dello spazio libero (dire ad esempio "**1fr 2fr**" significa dichiarare che la seconda è il doppio della prima).

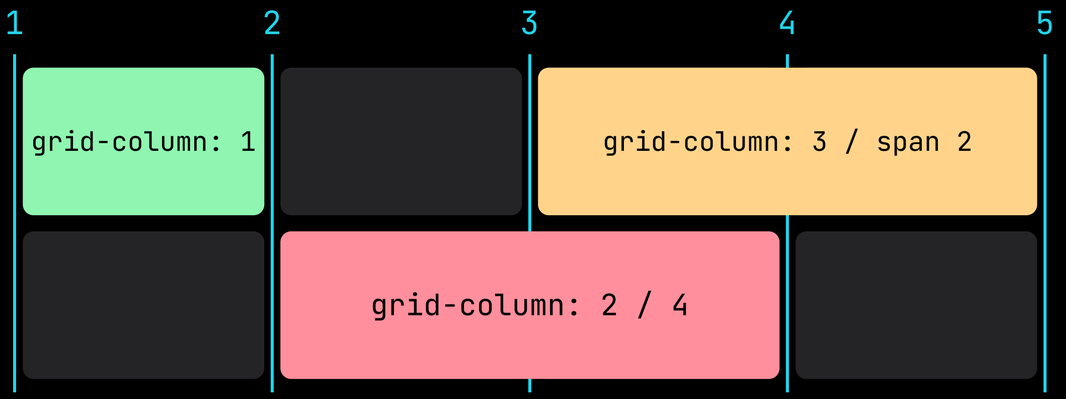
repeat(3, 1fr) → ripete una misura: equivale a "**1fr 1fr 1fr**".

gap → spazio tra le celle.

Le misure fisse **non sono obbligatorie**: **200px** è solo un esempio. Vanno bene anche **1fr 1fr 1fr**, **25% 50% 25%**, o un mix di unità.

Posizionare gli elementi

Una griglia con N colonne ha **N+1 linee**, numerate da 1. Con **grid-column** e **grid-row** diciamo a ogni elemento dove stare.



E se andiamo oltre? (griglia da 4 colonne)

`grid-column: 5;` → non è un errore: la grid **crea una colonna in più** (“implicita”), larga quanto il contenuto. Il layout si sballa.

`grid-row: 4;` con meno righe definite → nascono **righe implicite** (quelle vuote restano alte 0). Qui non è un problema: di solito si definiscono solo le colonne e le righe si creano da sole quando servono.

`grid-column: 1 / -1;` → i numeri negativi contano dalla fine: **-1 è l’ultima linea**, quindi l’elemento occupa tutta la riga.

Tre notazioni possibili:

- `grid-column: 1;`

  numero singolo: occupa quella colonna, cioè lo spazio tra la linea 1 e la linea 2

- `grid-column: 2 / 4;`

  inizio / fine: dalla linea 2 alla linea 4 (la fine è esclusa)

- `grid-column: 3 / span 2;`

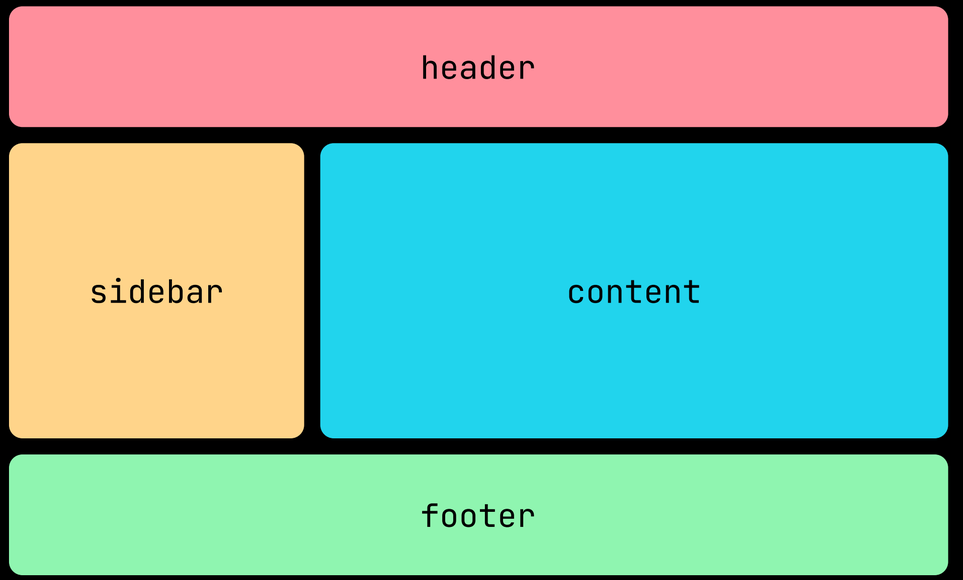
  inizio / span: dalla linea 3, si estende per 2 colonne (fino alla linea 5)

Named areas: dare un nome alle zone

In alternativa ai numeri, possiamo “disegnare” il layout con dei nomi, direttamente nel contenitore.

```css
.container {
    display: grid;
    grid-template-columns: 200px 1fr;
    grid-template-areas:
        "header  header"
        "sidebar content"
        "footer  footer";
}

.header  { grid-area: header; }
.sidebar { grid-area: sidebar; }
.content { grid-area: content; }
.footer  { grid-area: footer; }
```

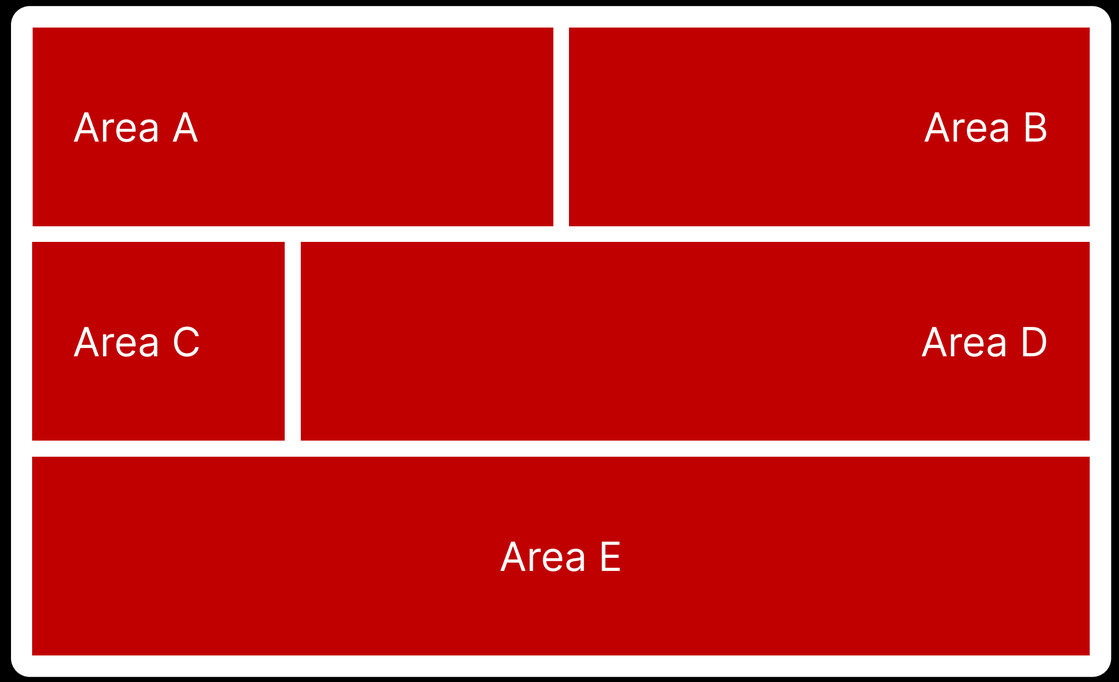


- ogni **stringa** è una **riga**, ogni **parola** è una **cella**
- ripetere un nome = l’area si estende su più celle
- ogni elemento si “aggancia” alla sua area con **grid-area**
- un punto (.) indica una cella vuota

Proviamo insieme: la tabella rossa

Scriviamo il CSS per visualizzare nel browser la griglia qui sotto:

- larghezza pari a quella del **viewport**
- misure esatte del gap e colori non rilevanti
- **due versioni**, entrambe con Grid:
  1. **senza** named areas (grid-column / grid-row)
  2. **con** named areas (grid-template-areas)



Tip: partite con `* { box-sizing: border-box; }` → padding e bordo vengono inclusi nella larghezza dell'elemento, invece di aggiungersi. Così `width: 100vw` con un po' di padding non "sfora" la pagina.

Normal grid vs named areas

Stesso risultato, due modi di scriverlo.

Normal grid

```
#a {
    grid-column: 1 / span 2;
    grid-row: 1;
}
#d {
    grid-column: 2 / span 3;
    grid-row: 2;
}
/* ...una regola per elemento */
```

Named areas

```
#main {
    grid-template-areas:
        "a a b b"
        "c d d d"
        "e e e e";
}
#a { grid-area: a; }
#d { grid-area: d; }
```

Normal grid → posizioni con i numeri delle linee: preciso e flessibile, ma il layout si “legge” solo mettendo insieme le regole di tutti gli elementi.

Named areas → il layout si vede a colpo d’occhio nel CSS ed è facile da cambiare: basta riscrivere le stringhe.

In entrambi i casi ogni elemento occupa sempre un **rettangolo**: può estendersi **su più colonne** e **su più righe** (es. `grid-row: 1 / span 2`, oppure `"sidebar header"` / `"sidebar content"`), ma non può formare una L. Con la *normal grid* una L non si può nemmeno scrivere; con le *named areas* sì, ma il browser **scarta l’intero** `grid-template-areas`.

Esercizio per casa

Ricreare il seguente layout. Non vi verrà fornito nulla se non l'immagine del gatto e alcune specifiche veloci.

Pagina con gatto Home About Services Contact

Perché i gatti fanno cosi?



I gatti hanno comportamenti affascinanti, e uno dei più curiosi è il gesto di mostrare la lingua, che molti proprietari interpretano come una smorfia buffa. Tuttavia, questo comportamento può avere diverse spiegazioni. Una delle ipotesi più comuni è il cosiddetto "Flehmen", un meccanismo attraverso il quale il gatto trasferisce feromoni e altre molecole dall'aria all'organo vomeronasale, situato nel palato. Questo processo aiuta il gatto a raccogliere informazioni sull'ambiente circostante.

Un'altra spiegazione potrebbe essere legata al rilassamento muscolare. I gatti, infatti, tendono a lasciar sporgere leggermente la lingua quando sono completamente rilassati o addormentati. In queste situazioni, i muscoli della bocca e della mandibola si allentano, causando talvolta l'esposizione della lingua. Questo comportamento è più frequente nelle razze brachicefale, come il Persiano, a causa della loro conformazione facciale.

In alcuni casi, il mostrare la lingua potrebbe anche essere un segno di disagio fisico, soprattutto se accompagnato da altri sintomi. Problemi dentali o infezioni orali possono indurre il gatto a mantenere la lingua fuori dalla bocca. È importante, quindi, osservare questo comportamento nel contesto generale della salute del gatto e, se necessario, consultare un veterinario.

Infine, alcuni gatti sviluppano questo comportamento come abitudine o peculiarità individuale, senza che ci sia una ragione medica o funzionale precisa. La varietà di motivazioni dietro il gesto della linguaccia rende questo comportamento tanto interessante quanto misterioso per gli studiosi del comportamento felino.

Related Articles

- 10 Motivi per cui il tuo Gatto fa la Linguaccia
- Come Interpretare le Espressioni del tuo Micio
- Gatti Buffi: Le Migliori Smorfie

Contattaci: email@example.com | +123 456 7890

Twitter | Facebook | LinkedIn

Specifiche

- Utilizzare **FLEX** layout;
- Nel creare la pagina HTML, vanno utilizzati sia una sezione ***Article*** che una sezione ***Aside*** (per la sidebar);
- Sidebar e Articolo occupano rispettivamente **1/3** e **2/3** dello spazio disponibile;
- Rendere attivo (almeno) il link alla pagina dell'***About***, che vi è stata fornita;
- Utilizzare effetti di ***hover*** sui link;
- In sidebar sembrano esserci tre bottoni. **NON sono bottoni**. Cercate di ricreare uno stile simile;
- Il testo va giustificato.

In aggiunta, se siete in grado, provate a creare un design responsive con una semplice ***media-query***:

*Se lo schermo ha una larghezza di **768px** o inferiore, gli elementi principali del design flex (main, header, footer) cambiano disposizione e si mettono in colonna. Per bordi e margini non vi preoccupate.*

E poi la pagina About

Una volta finita la pagina con il gatto, create uno stile anche per la **pagina di About** che vi ho fornito (**provate ad utilizzare anche le grid qui!**). La struttura è pressoché simile a quella dell'index, quindi non dovreste avere particolari problemi.

Grazie a tutti :)
Buona giornata!